

Best Programming Practice

1. Use Variables including for Fixed, User Inputs, and Results
2. Use Methods instead of writing code in the main() function
3. Proper naming conventions for all variables and methods
4. Proper Program Name and Class Name
5. Handle Checked and Unchecked Exceptions wherever possible
6. Proper Method Name which indicates action taking inputs and providing result

Sample Program 1: Create a program to find all the occurrences of a character in a string using charAt() method

- a. Take user input for the String and occurrences of the Character to find
- b. Write a method to find all the occurrences of the characters.
 - i. The logic used is to first find the number of occurrences of the character and
 - ii. then create an array to store the indexes of the character
- c. Call the method in the main and display the result

Java

```
// Program to find all the occurrences of a character in a string
import java.util.Scanner;
class StringAnalyzer {
    // Method to find all the index of a character in a string using charAt()
    // method and return them in an array
    public static int[] findAllIndexes(String text, char ch) {
        // The count is used to find the number of occurrences of the character
        int count = 0;
        for (int i = 0; i < text.length(); i++) {
            if (text.charAt(i) == ch) {
                count++;
            }
        }

        // Create an array to store the indexes of the character
        int[] indexes = new int[count];
        int j = 0;
        for (int i = 0; i < text.length(); i++) {
            if (text.charAt(i) == ch) {
                indexes[j] = i;
                j++;
            }
        }
        return indexes;
    }
}
```

```
public static void main(String[] args) {  
    // Take user input for Text and Character to check Occurrences  
    Scanner sc = new Scanner(System.in);  
    System.out.print(Enter a text: ");  
    String text = sc.nextLine();  
    System.out.print("Enter a character to find the occurrences: ");  
    char ch = sc.next().charAt(0);  
  
    // Find the occurrences of the character  
    int[] indexes = findAllIndexes(text, ch);  
  
    // Display the occurrences of the character  
    System.out.println("Indexes of the character '" + ch + "': ");  
    for (int i = 0; i < indexes.length; i++) {  
        System.out.print(indexes[i] + " ");  
    }  
}
```

Level 1 Practice Programs

1. Write a program to compare two strings using the `charAt()` method and check the result with the built-in String `equals()` method

Hint =>

- a. Take user input using the `Scanner next()` method for 2 String variables
- b. Write a method to compare two strings using the `charAt()` method and return a boolean result
- c. Use the String Built-In method to check if the results are the same and display the result

```
import java.util.Scanner;
public class StringComparison {
    public static void main(String[] args) {

        //creating input object to take inputs
        Scanner input = new Scanner(System.in);

        //taking user inputs for string str1
        System.out.print("Enter a String : ");
        String str1 = input.next();

        //taking user inputs for string str2
        System.out.print("Enter a String : ");
        String str2 = input.next();

        //comparing the strings using charAt
        boolean compareByCharAt = compareUsingCharAt(str1, str2);

        //comparing the strings using builtin equals() method
        boolean compareByMethod = str1.equals(str2);

        //printing the results
        if(compareByCharAt) {
            System.out.println(str1 + " and " + str2 + " are same on
comparing using charAt.");
        } else {
            System.out.println(str1 + " and " + str2 + " are
different on comparing using charAt.");
        }
        if(compareByMethod) {
```

```

        System.out.println(str1 + " and " + str2 + " are same on
comparing using builtin equals method.");
    } else {
        System.out.println(str1 + " and " + str2 + " are
different on comparing using builtin equals method.");
    }
    //closing the input
    input.close();
}
//method to compare two strings using charAt()
public static boolean compareUsingCharAt(String str1, String str2) {

    //returning false if length are different
    if(str1.length() != str2.length()) {
        return false;
    }

    //returning false if character differ
    for(int i=0; i<str1.length(); i++) {
        if (str1.charAt(i) != str2.charAt(i)) {
            return false;
        }
    }
    //returning true because strings are same
    return true;
}
}

```

2. Write a program to create a substring from a String using the **charAt()** method. Also, use the String built-in method **substring()** to find the substring of the text. Finally Compare the the two strings and display the results

Hint =>

- a. Take user input using the **Scanner next()** method to take the String variable and also the start and the end index to get the substring from the given text
- b. Write a method to create a substring from a string using the **charAt()** method with the string, start, and end index as the parameters
- c. Write a method to compare two strings using the charAt() method and return a boolean result

- d. Use the String built-in method substring() to get the substring and compare the two strings. And finally display the result

```
import java.util.Scanner;
public class SubstringFinder {
    public static void main(String[] args) {

        //creating input object to take inputs
        Scanner input = new Scanner(System.in);

        //taking user inputs for string str
        System.out.print("Enter a String : ");
        String str = input.next();

        //taking user inputs for starting and ending indexes
        System.out.print("Enter the starting index : ");
        int start = input.nextInt();
        System.out.print("Enter the ending index : ");
        int end = input.nextInt();

        //creating variable substringByCharAt to store substring using
        charAt
        String substringByCharAt = substringUsingCharAt(str, start,
        end);

        //creating variable substringByMethod to store substring using
        equals method
        String substringByMethod = str.substring(start-1, end-1);

        //printing the results
        System.out.println("Substring of " + str + " from index " +
        start + " to " + end + " is " + substringByCharAt + " using charAt");
        System.out.println("Substring of " + str + " from index " +
        start + " to " + end + " is " + substringByMethod + " using builtin
        substring method");

        //closing the input
        input.close();
    }
    //method to find substring using charAt()
    public static String substringUsingCharAt(String str, int start, int
    end) {
```

```
//creating tempArray array to store character array of
character of substring
char[] tempArray = new char[end - start+1];

//extracting characters from string
for(int i=start-1; i<end-1; i++) {
    tempArray[i-start+1] = str.charAt(i);
}
//creating string subString to store substring
String subString = new String(tempArray);

//returning the substring
return subString;
}
}
```

3. Write a program to return all the characters in a string using the user-defined method, compare the result with the String built-in toCharArray() method, and display the result

Hint =>

- a. Take user input using the **Scanner next()** method to take the text into a String variable
- b. Write a method to return the characters in a string without using the **toCharArray()**
- c. Write a method to compare two string arrays and return a boolean result
- d. In the main() call the user-defined method and the String built-in toCharArray() method, compare the 2 arrays, and finally display the result

```
import java.util.Scanner;
public class CharacterExtractor {
    public static void main(String[] args) {

        //creating input object to take inputs
        Scanner input = new Scanner(System.in);

        //taking user inputs for string str
        System.out.print("Enter a String : ");
        String str = input.next();

        //creating array charArray1 to store characters of string using
        charAt
        char[] charArray1 = findCharacters(str);
```

```

        //creating array charArray2 to store characters of string using
toCharArray Method
        char[] charArray2 = str.toCharArray();

        //printing the results
        System.out.println("Characters of " + str + " using charAt are
:");

        for(int i=0;i<charArray1.length; i++) {
            System.out.print(" " + charArray1[i]);
        }
        System.out.println("\nCharacters of " + str + " using
toCharArray Method are : ");
        for(int i=0;i<charArray1.length; i++) {
            System.out.print(" " + charArray2[i]);
        }
        //closing the input
        input.close();
    }
    //method to find characters of string using charAt()
    public static char[] findCharacters(String str) {

        //creating charArray array to store character array of
character of string
        char[] charArray = new char[str.length()];

        //extracting characters from string
        for(int i=0; i<str.length(); i++) {
            charArray[i] = str.charAt(i);
        }
        //returning the substring
        return charArray;
    }
}

```

4. Write a program to demonstrate NullPointerException.

Hint =>

- a. Write a Method to generate the Exception. Here define the variable text and initialize it to null. Then call one of the String Method to generate the exception

- e. Write the Method to demonstrate **NullPointerException**. Here define the variable text and initialize it to null. Then write try catch block for handling the Exception while accessing one of the **String** method
- b. From the main Firstly call the method to generate the Exception then refactor the code to call the method to handle the RuntimeException

```
public class NullPointerExceptionHandler {
    public static void main(String[] args) {
        System.out.println("Demonstrating NullPointerException generation:");
    };

    try {
        // This will throw the exception
        generateException();
    } catch (NullPointerException e) {
        System.out.println("Exception caught in main method: " +
e.getMessage());
    }

    System.out.println("\nDemonstrating handling of
NullPointerException: ");

    // This handles the exception
    handleException();
}

// Method to generate the NullPointerException
public static void generateException() {
    // Initialize text to null
    String text = null;

    // Attempt to call a String method on null to generate the
exception
    System.out.println("Length of text: " + text.length());
}

// Method to handle the NullPointerException
public static void handleException() {
    // Initialize text to null
    String text = null;
    try {
        // Attempt to call a String method on null
        System.out.println("Length of text: " + text.length());
    }
}
```

```

        } catch (NullPointerException e) {
            System.out.println("NullPointerException caught! Text is
null.");
        }
    }
}

```

5. Write a program to demonstrate **StringIndexOutOfBoundsException**

Hint =>

- Define a variable of type String and take user input to assign a value
- Write a Method to generate the Exception. Access the index using `charAt()` beyond the length of the String. This will generate a runtime exception and abruptly stop the program.
- Write the Method to demonstrate **StringIndexOutOfBoundsException**. Access the index using **`charAt()`** beyond the length of the String. Then write try catch block for Exception while accessing the String method
- From the main Firstly call the method to generate the Exception then call the method to handle the RuntimeException

```

import java.util.Scanner;

public class StringIndexOutOfBoundsExceptionHandler {

    public static void main(String[] args) {

        //creating scanner object to take input
        Scanner input = new Scanner(System.in);

        // Taking user input
        System.out.print("Enter a string: ");
        String inputString = input.nextLine();

        // Calling the method to generate the exception
        System.out.println("\nDemonstrating exception generation: ");
        try {
            generateException(inputString);
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("Exception caught in main: " +
e.getMessage());
        }
    }
}

```

```
// Calling the method to handle the exception
System.out.println("\nDemonstrating exception handling:");
handleException(inputString);
}

// Method to generate StringIndexOutOfBoundsException
public static void generateException(String inputString) {
    System.out.println("Generating
StringIndexOutOfBoundsException...");

    // Accessing index beyond the length of the string
    System.out.println("Character at index 10: " +
inputString.charAt(10));
}

// Method to handle StringIndexOutOfBoundsException
public static void handleException(String inputString) {

    System.out.println("Handling
StringIndexOutOfBoundsException...");
    try {

        // Accessing index beyond the length of the string
        System.out.println("Character at index 10: " +
inputString.charAt(10));

        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("StringIndexOutOfBoundsException caught!
String length is less than 10.");
        }
    }
}
```

6. Write a program to demonstrate *IllegalArgumentExce*

Hint =>

- Define a variable of type String and take user input to assign a value
- Write a Method to generate the Exception. Here use the *subString()* and set the start index to be greater than the end index. This will generate a runtime exception and abruptly stop the program.

- c. Write the Method to demonstrate ***IllegalArgumentException***. Here use the ***substring()*** and set the start index to be greater than the end index. This will generate a runtime exception. Use the try-catch block to handle the ***IllegalArgumentException*** and the generic runtime exception
- d. From the main Firstly call the method to generate the Exception then call the method to handle the RuntimeException

```
import java.util.Scanner;

public class IllegalArgumentExceptionHandler {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Taking user input
        System.out.print("Enter a string: ");
        String inputString = input.nextLine();

        // Calling the method to generate the exception
        System.out.println("\nDemonstrating exception generation:");
        try {
            generateException(inputString);
        } catch (IllegalArgumentException e) {
            System.out.println("Exception caught in main: " +
e.getMessage());
        } catch (RuntimeException e) {
            System.out.println("Generic Exception exception caught in main:
" + e.getMessage());
        }

        // Calling the method to handle the exception
        System.out.println("\nDemonstrating exception handling:");
        handleException(inputString);
    }

    // Method to generate IllegalArgumentException
    public static void generateException(String inputString) {
        System.out.println("Generating IllegalArgumentException...");

        // Using substring() with start index greater than end index
        String result = inputString.substring(5, 2);
        System.out.println("Substring: " + result);
    }
}
```

```

    }

    // Method to handle IllegalArgumentException
    public static void handleException(String inputString) {
        System.out.println("Handling IllegalArgumentException...");
        try {
            // Using substring() with start index greater than end index
            String result = inputString.substring(5, 2);
            System.out.println("Substring: " + result);
        } catch (IllegalArgumentException e) {
            System.out.println("Caught IllegalArgumentException: Enter
valid range.");
        } catch (RuntimeException e) {
            System.out.println("Caught RuntimeException: Enter valid
range.");
        }
    }
}

```

7. Write a program to demonstrate **NumberFormatException**

Hint =>

- Define a variable to take user input as a String
- Use Integer.parseInt() to generate this exception. **Integer.parseInt()** is a built-in function in java.lang.Integer class to extract the number from text. In case the text does not contain numbers the method will throw NumberFormatException which is a runtime exception
- Write a Method to generate the Exception. Use **Integer.parseInt(text)** to extract number from the text. This will generate a runtime exception and abruptly stop the program.
- Write the Method to demonstrate **NumberFormatException**. Use **Integer.parseInt(text)** to extract number from the text. This will generate a runtime exception. Use the try-catch block to handle the **NumberFormatException** as well as the generic runtime exception
- From the main Firstly call the method to generate the Exception then call the method to handle the RuntimeException

```

import java.util.Scanner;

public class NumberFormatExceptionHandler {

```

```
// Method to generate NumberFormatException
public static void generateException(String inputString) {
    System.out.println("Generating NumberFormatException...");
    // Use Integer.parseInt to convert string to integer
    int number = Integer.parseInt(inputString);
    System.out.println("Parsed number: " + number);
}

// Method to handle NumberFormatException
public static void handleException(String inputString) {
    System.out.println("Handling NumberFormatException...");
    try {
        // Use Integer.parseInt to convert string to integer
        int number = Integer.parseInt(inputString);
        System.out.println("Parsed number: " + number);
    } catch (NumberFormatException e) {
        System.out.println("Caught NumberFormatException: Accepting
only integers.");
    } catch (RuntimeException e) {
        System.out.println("Caught RuntimeException: Accepting only
integers");
    }
}

public static void main(String[] args) {
    Scanner input = new Scanner(System.in);

    // Taking user input
    System.out.print("Enter a string: ");
    String inputString = input.nextLine();

    // Calling the method to generate the exception
    System.out.println("\nDemonstrating exception generation:");
    try {
        generateException(inputString);
    } catch (NumberFormatException e) {
        System.out.println("Exception caught in main: " +
e.getMessage());
    } catch (RuntimeException e) {
        System.out.println("Generic exception caught in main: " +
e.getMessage());
    }
}
```

```

    }

    // Calling the method to handle the exception
    System.out.println("\nDemonstrating exception handling:");
    handleException(inputString);
}
}

```

8. Write a program to demonstrate **ArrayIndexOutOfBoundsException**

Hint =>

- Define a variable of array of names and take input from the user
- Write a Method to generate the Exception. Here access index larger then the length of the array. This will generate a runtime exception and abruptly stop the program.
- Write the Method to demonstrate **ArrayIndexOutOfBoundsException**. Here access index larger then the length of the array. This will generate a runtime exception. Use the try-catch block to handle the **ArrayIndexOutOfBoundsException** and the generic runtime exception
- From the main Firstly call the method to generate the Exception then call the method to handle the RuntimeException

```

import java.util.Scanner;

public class ArrayIndexOutOfBoundsExceptionHandler {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // creating an array of names and take input from the user
        System.out.print("Enter the size of the array: ");
        int size = input.nextInt();

        //new line
        input.nextLine();

        String[] names = new String[size];
        for (int i = 0; i < size; i++) {
            System.out.print("Enter name " + (i + 1) + ": ");
            names[i] = input.nextLine();
        }
    }
}

```

```
// Calling the method to generate the exception
System.out.println("\nDemonstrating exception generation:");
try {
    generateException(names);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Exception caught in main: " +
e.getMessage());
} catch (RuntimeException e) {
    System.out.println("Generic exception caught in main: " +
e.getMessage());
}

// Call the method to handle the exception
System.out.println("\nDemonstrating exception handling:");
handleException(names);
}

// Method to generate ArrayIndexOutOfBoundsException
public static void generateException(String[] names) {
    System.out.println("Generating ArrayIndexOutOfBoundsException...");

    // Attempting to access an index beyond the array's length
    System.out.println("Accessing index 10: " + names[10]);
}

// Method to handle ArrayIndexOutOfBoundsException
public static void handleException(String[] names) {
    System.out.println("Handling ArrayIndexOutOfBoundsException...");
    try {
        // Attempting to access an index beyond the array's length
        System.out.println("Accessing index 10: " + names[10]);
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Caught ArrayIndexOutOfBoundsException:
Length of array is less than 10. ");
    } catch (RuntimeException e) {
        System.out.println("Caught RuntimeException: Length of array is
less than 10.");
    }
}
}
```

9. Write a program to convert the complete text to uppercase and compare the results

Hint =>

- Take user input using the **Scanner** **nextLine()** method to take the complete text into a String variable
- Write a method using the String built-in **charAt()** method to convert each character if it is lowercase to the Upper Case. Use the logic ASCII value of 'a' is 97 and 'A' is 65 so the difference is 32, similarly ASCII value of 'b' is 98 and 'B' is 66 so the difference is 32, and so on
- Write a method to compare two strings using the charAt() method and return a boolean result
- In the main() use the String built-in method **toLowerCase()** to get the Uppercase Text and compare the two strings using the user-defined method. And finally display the result

```
import java.util.Scanner;

public class UppercaseComparison {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Taking input from the user
        System.out.print("Enter the text: ");
        String inputText = scanner.nextLine();

        // Converting the input text to uppercase using the built-in method
        String builtInUppercase = inputText.toUpperCase();

        // Converting the input text to uppercase using the custom method
        (ASCII) String customUppercase = convertToUpperCaseUsingAscii(inputText);

        // Comparing both strings using the user-defined method
        boolean isEqual = compareStringsUsingCharAt(builtInUppercase,
        customUppercase);

        // Displaying the results
        System.out.println("Original Text: " + inputText);
        System.out.println("Uppercase using toUpperCase(): " +
        builtInUppercase);
        System.out.println("Uppercase using custom method: " +
```

```
customUppercase);
    System.out.println("Are both results equal? " + isEqual);

    scanner.close();
}
// Method to convert the string to uppercase using ASCII values
public static String convertToUpperCaseUsingAscii(String text) {
    StringBuilder result = new StringBuilder();

    // Loop through each character in the input text
    for (int i = 0; i < text.length(); i++) {
        char ch = text.charAt(i);

        // Checking if the character is lowercase
        if (ch >= 'a' && ch <= 'z') {
            // Converting it to uppercase by adjusting ASCII value
            ch = (char) (ch - 32);
        }

        // Adding the character to the result
        result.append(ch);
    }
    // Returning the converted string
    return result.toString();
}

// Method to compare two strings using charAt() method
public static boolean compareStringsUsingCharAt(String str1, String
str2) {
    if (str1.length() != str2.length()) {
        // If lengths are not the same, they are not equal
        return false;
    }

    for (int i = 0; i < str1.length(); i++) {
        if (str1.charAt(i) != str2.charAt(i)) {
            // If any character doesn't match, the strings are not
            equal

            return false;
        }
    }
    // If all characters match, the strings are equal
}
```

```
        return true;
    }
}
```

10. Write a program to convert the complete text to lowercase and compare the results

Hint =>

- Take user input using the **Scanner nextLine()** method to take the complete text into a String variable
- Write a method using the String built-in **charAt()** method to convert each character if it is lowercase to the Upper Case. Use the logic ASCII value of 'a' is 97 and 'A' is 65 so the difference is 32, similarly ASCII value of 'b' is 98 and 'B' is 66 so the difference is 32, and so on
- Write a method to compare two strings using the charAt() method and return a boolean result
- In the main() use the String built-in method **toUpperCase()** to get the Uppercase Text and compare the two strings using the user-defined method. And finally display the result

```
import java.util.Scanner;

public class LowercaseComparison {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Taking input from the user
        System.out.print("Enter the text: ");
        String inputText = scanner.nextLine();

        // Converting the input text to lowercase using the built-in method
        String builtInLowercase = inputText.toLowerCase();

        // Converting the input text to lowercase using the custom method
        (ASCII) String customLowercase = convertToLowercaseUsingAscii(inputText);

        // Comparing both strings using the user-defined method
        boolean isEqual = compareStringsUsingCharAt(builtInLowercase,
        customLowercase);
    }
}
```

```
// Displaying the results
System.out.println("Original Text: " + inputText);
System.out.println("Lowercase using toLowerCase(): " +
builtInLowercase);
System.out.println("Lowercase using custom method: " +
customLowercase);
System.out.println("Are both results equal? " + isEqual);

// Splitting the text into words and getting their lengths
String[][] wordLengthArray =
splitTextIntoWordsAndLengths(inputText);

// Displaying the words and their lengths
System.out.println("\nWords and their lengths:");
for (int i = 0; i < wordLengthArray.length; i++) {
    System.out.println("Word: " + wordLengthArray[i][0] + ",
Length: " + wordLengthArray[i][1]);
}

scanner.close();
}

// Method to convert the string to lowercase using ASCII values
public static String convertToLowerCaseUsingAscii(String text) {
    StringBuilder result = new StringBuilder();

    // Loop through each character in the input text
    for (int i = 0; i < text.length(); i++) {
        char ch = text.charAt(i);

        // Check if the character is uppercase
        if (ch >= 'A' && ch <= 'Z') {
            // Converting it to lowercase by adjusting the ASCII value
            ch = (char) (ch + 32);
        }

        // Adding the character to the result
        result.append(ch);
    }

    // Returning the converted string
    return result.toString();
}
```

```
// Method to compare two strings using charAt() method
public static boolean compareStringsUsingCharAt(String str1, String
str2) {
    if (str1.length() != str2.length()) {
        // If lengths are not the same, they are not equal
        return false;
    }

    for (int i = 0; i < str1.length(); i++) {
        if (str1.charAt(i) != str2.charAt(i)) {
            // If any character doesn't match, the strings are not
equal
            return false;
        }
    }

    return true; // If all characters match, the strings are equal
}

// Method to split the text into words and return a 2D array with words
and their lengths
public static String[][] splitTextIntoWordsAndLengths(String text) {
    // Split the input text into words using space as a delimiter
    String[] words = text.split(" ");

    // Create a 2D array to store words and their lengths
    String[][] wordLengthArray = new String[words.length][2];

    for (int i = 0; i < words.length; i++) {
        wordLengthArray[i][0] = words[i]; // Store the word
        wordLengthArray[i][1] = String.valueOf(words[i].length()); //
Store the word length
    }

    return wordLengthArray; // Return the 2D array
}
}
```